Symbols and Glossary

Symbols and Glossary I

This glossary defines the Generalized Notation Notation (GNN) language constructs used to specify generative models, together with the Active Inference symbols those constructs carry across the model kinds of sec. 3.2. Definitions follow the GNN syntax specification and the exemplar specifications distributed with the repository — the discrete POMDP exemplars and the continuous linear-Gaussian exemplars alike — and they align with the standard formulations of both kinds (Smékal and Friedman 2023; Da Costa et al. 2020; Parr et al. 2022).

Language Constructs I

A GNN file is an ordered, UTF-8 Markdown document whose level-2 headers name required and optional sections. The strict schema validator enforces section order, declaration grammar, and connection syntax; the constructs below are the load-bearing sections a parser must recognize; they are catalogued in tbl. 1.

Language Constructs I

Table 1: The GNN language constructs a conforming parser must recognize, with the meaning each carries. The first rows are the required sections that pin a model's identity, variables, and factor graph; the optional sections that follow carry parameterization, ontology bindings, timing, equations, and provenance. The operator rows are the connection vocabulary the `Connections` section is written in — directed `A>B`, undirected `A-B`, and their labeled v1.1 forms — and `default=…` supplies initialization hints. Definitions follow the GNN syntax specification and the exemplar specifications under `input/gnn_files`, and the strict schema validator enforces section order and declaration grammar before any downstream step runs. The construct vocabulary is deliberately kind-agnostic: the same `StateSpaceBlock`, `Connections`, and operator syntax declare a discrete categorical model, a continuous linear-Gaussian model, or a composed multi-agent or hierarchical one — what differs between kinds is which matrix names the block declares, as tbl. 2 details.

Construct	Meaning
`## GNNSection`	Required short identifier for the model block (no spaces, e.g. `ActInfPOMDP`).
`## GNNVersionAndFlags`	Required version declaration (`GNN v1` or `GNN v1.1`) with optional flags.
`## ModelName`	Required human-readable title of the generative model.
`## StateSpaceBlock`	Required block declaring every variable and matrix as `NAME[dim, …, type=…]`, one per line — the block whose declared vocabulary fixes the model's kind.
`## Connections`	Required edge list relating state-space variables, expressing the model's factor graph.
`## ModelAnnotation`	Optional free-text description of the model's modalities, factors, and assumptions.
`## InitialParameterization`	Optional concrete numeric values for declared matrices and vectors.
`## ActInfOntologyAnnotation`	Optional bindings from each variable to a CamelCase Active Inference ontology term.
`## ModelParameters`	Optional key-value dimensions (e.g. `num_hidden_states`, `num_obs`, `num_actions`) consumed by code generators.
`## Time`	Optional dynamics declaration: a time variable plus `Dynamic/Static`, `Discrete/Continuous`, and `ModelTimeHorizon`.
`## Equations`	Optional LaTeX-rendered formulas defining model dynamics and the relationships between declared variables; round-tripped by the semantic-fidelity gate.

Language Constructs II

`## Footer`	Optional closing section that terminates the file and lets a reader or parser enter it from either end.
`## Signature`	Optional provenance block carrying a cryptographic signature over the specification.
`NAME[d ,d ,…,type=…]`	A variable or tensor declaration; dimensions are positive integers or named references, and a `type` (`float`, `int`, `bool`) is required.
`A>B`	Directed (causal) connection operator: edge from `A` to `B`, e.g. `D>s` (a prior conditions a hidden state) or `F>x` (a transition matrix drives a continuous state).
`A-B`	Undirected (bidirectional) connection operator, e.g. `s-A` (a hidden state participates in the likelihood mapping).
`A>B:label` / `A-B:label`	A v1.1 annotated edge; the trailing label documents the relation and is preserved but may be ignored for structural validation.
`default=…`	A v1.1 declaration hint (`uniform`, `zeros`, `ones`, `eye`, `random`) supplying an initialization for a matrix or vector.

Symbols by Model Kind I

The exemplar specifications declare the generative-model components of Active Inference, mapping each GNN variable to its probabilistic meaning (Da Costa et al. 2020; Smith et al. 2022; Parr et al. 2022). Because the notation spans model kinds, the same letter can name different objects in different kinds — most prominently `F`, which is the variational free energy of eq. 6 in the discrete kind and the state-transition matrix of eq. 9 in the continuous kind — so each row of tbl. 2 names the kind its reading belongs to and the equation in sec. 3 that fixes it. These authored cross-references support review of agreement between the glossary and formal statement; their consistency still requires validation when either changes.

Symbols by Model Kind I

Table 2: The Active Inference symbols a GNN specification carries, scoped by model kind and bound to the equation of sec. 3 that fixes each role. Read the discrete-kind rows as the categorical generative-model tensors (likelihood, controlled transitions, log-preferences, prior over initial states, habit prior) with `s`, `o`, π, and `u` as the inference variables and `F` and `G` as the two free-energy functionals minimized at eq. 6 and eq. 7; read the continuous-kind rows as the linear-Gaussian parameterization — system matrices, covariances, Gaussian prior, and the optional closed-loop declarations of eq. 11. The rows where one letter carries two kind-scoped readings (`F`, `u`, `t`) are deliberate: the notation reuses a compact alphabet across kinds, and the declared matrix vocabulary of the `StateSpaceBlock` — not the letter alone — is what fixes the reading, which is exactly what the kind classification of sec. 3.2 computes. Use the table as a decoding key when reading a `StateSpaceBlock`: every declared matrix name should map to exactly one kind-scoped row here, and the `## ActInfOntologyAnnotation` bindings described below are what make that mapping machine-checkable downstream.

Symbol	Meaning
`A`	Discrete kind: likelihood (observation) matrix encoding $P(o \mid s)$, a column-stochastic map from hidden states to observation outcomes (eq. 2).
`B`	Discrete kind: transition tensor encoding $P(s' \mid s, u)$, one column-stochastic slice per action in the canonical `B[next_state][previous_state][action]` order (eq. 3).
`C`	Discrete kind: preference vector — log-preferences over observation outcomes that bias the agent toward preferred outcomes (eq. 4).
`D`	Discrete kind: prior vector over initial hidden states (eq. 5).
`E`	Discrete kind: habit vector — an initial policy prior over actions, entering the policy posterior (eq. 8).
`s`	Discrete kind: current hidden-state distribution; `s_prime` (`s'`) is the next hidden-state distribution.
`o`	Discrete kind: current observation, an integer index over outcome modalities.
π	Discrete kind: policy — a distribution over actions inferred from expected free energy (eq. 8).
`u`	Discrete kind: the selected (sampled) action, sampled from the policy posterior. Continuous kind: the control input added to the state each step (eq. 9).

Symbols by Model Kind II

`F`	Discrete kind: variational free energy, minimized during state inference (eq. 6) (Friston 2010). Continuous kind: the state-transition matrix of the linear-Gaussian model (eq. 9).
`G`	Discrete kind: expected free energy per policy, minimized during policy inference to score candidate actions (eq. 7) (Da Costa et al. 2020).
`t`	Discrete time step; the horizon T bounds the product in eq. 1. Continuous kind: the time index of the state-space model.
`H`	Continuous kind: observation matrix mapping the latent state to the observation (eq. 10).
`Q`	Continuous kind: process-noise covariance of the state evolution (eq. 9).
`R`	Continuous kind: observation-noise covariance of the readout (eq. 10).
`x`	Continuous kind: the Gaussian latent state x_t; `x[2,1,type=float]` in the navigation exemplar declares a two-dimensional position.
`y`	Continuous kind: the continuous observation read out linearly from the state (eq. 10).
`prior_mean`	Continuous kind: prior mean μ_0 over the initial latent state (eq. 9).
`prior_cov`	Continuous kind: prior covariance Σ_0 over the initial latent state (eq. 9).
`goal_mean`	Continuous kind: preferred state $\mu^\star$ the closed-loop controller steers toward (eq. 11).
`control_gain`	Continuous kind: scalar proportional gain k on the belief error in the closed loop (eq. 11).

Ontology Bindings and Implementations I

The `## ActInfOntologyAnnotation` section binds each variable to a canonical term — `A=LikelihoodMatrix`, `B=TransitionMatrix`, `C=LogPreferenceVector`, `D=PriorOverHiddenStates`, `s=HiddenState`, `o=Observation`, π`=PolicyVector` in the discrete exemplars; `F=StateTransitionMatrix`, `H=ObservationMatrix`, `Q=ProcessNoiseCovariance`, `R=ObservationNoiseCovariance`, `prior_mean=PriorMean`, `prior_cov=PriorCovariance`, `goal_mean=PreferredState`, `control_gain=ControlGain`, `x=ContinuousHiddenState`, `y=ContinuousObservation`, `u=ControlInput` in the continuous exemplars — which downstream pipeline steps use for semantic analysis and validation. The bindings are per-kind by

construction: a continuous exemplar binds `F` to `StateTransition Matrix`, never to a free-energy term, so the ontology layer carries the same kind-scoped reading tbl. 2 fixes by hand. The same GNN specification feeds the project's rendering backends — including the 12 registered targets (PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn), of which the 12 render targets with execution adapters (PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn) can be selected at Step 12 subject to model-kind admission and runtime readiness — with per-kind reach recorded as explicit statuses (tbl. 4) — so that a model written once in this notation can be parsed, visualized, and executed across the 25-step pipeline (0–24) without restating its

mathematics (Smékal and Friedman 2023; Heins et al. 2022; Felice et al. 2021).